```
function [calfactors] =
ps_calibration(filenames_slc,filenames_ifgs,filename_water_mask,psc_selection_gr
idsize,slc_selection,crop,crop_in)

% Function to calibrate a stack of co-registered slc's
%
% Input:    - filenames_slc           filenames for calibrated slc's
%           - filenames_ifgs          filenames for interferograms
%           - filename_water_mask     mask for water areas, 'filename' or []
%           - psc_selection_gridsize  gridsize [m]
%           - slc_selection           selection of slc's used to select psc's
%           - crop                    borders of final crop
%           - crop_in                 borders of input crops
%
% Output:   - calfactors              calibration factors
%
% ------------------------------------------------------------------------
% File.............: ps_calibration.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Freek van Leijen
%                   Gini Ketelaar
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log



% ------------------------------------------------------------------------
% Initialize
% ------------------------------------------------------------------------

global max_mem_buffer Nlines Npixels az_spacing r_spacing fig
global ps_eval_method detail_plots project_id

Nifgs = size(filenames_ifgs,1);
Nslc = size(filenames_slc,1);
Nslc_selected = length(slc_selection);
Ncal = 1000;



% ------------------------------------------------------------------------
% Determine buffersize
% ------------------------------------------------------------------------

Nlines_g = round(psc_selection_gridsize/az_spacing);
Npixels_g = round(psc_selection_gridsize/r_spacing);

Ng_az = floor(Nlines/Nlines_g);
Ng_r = floor(Npixels/Npixels_g);
offset_az = floor(rem(Nlines,Nlines_g)/2);
offset_r = floor(rem(Npixels,Npixels_g)/2);

buffer_size_az = floor(max_mem_buffer/(8*8*Nifgs*Npixels*Nlines_g));
% buffer_size_az in number of gridcells in azimuth direction
% 6 because of 2 float array's, 2 complex array's, 1 multi image
```

```
% and 1 for the rest, 8 because of double

if (buffer_size_az == 0)
  buffer_size_az = 1;
  buffer_size_r = floor(max_mem_buffer/(8*8*Nifgs*Npixels_g*Nlines_g));
  if (buffer_size_r == 0)
    error('Not enough memory to proceed. Please increase the allocated memory or
reduce the grid size');
  end
  Nbuffers_r = floor(Ng_r/buffer_size_r);
  rem_buffer_r = rem(Ng_r,buffer_size_r);
else
  buffer_size_r = Ng_r;
  Nbuffers_r = 1;
  rem_buffer_r = 0;
end

if (buffer_size_az*Nlines_g > Nlines)
  buffer_size_az = Ng_az;
end

Nbuffers_az = floor(Ng_az/buffer_size_az);
rem_buffer_az = rem(Ng_az,buffer_size_az);



% ------------------------------------------------------------------------
% Determine borders gridcells
% ------------------------------------------------------------------------

grid_array_az = repmat(NaN,Ng_az,3);
grid_array_az(:,1) = [1 offset_az+Nlines_g+1:Nlines_g:offset_az+(Ng_az-
1)*Nlines_g+1]';
grid_array_az(:,2) = [offset_az+Nlines_g:Nlines_g:offset_az+ ...
                      (Ng_az-1)*Nlines_g Nlines]';
for v = 1:Nbuffers_az
  buffer = (v-1)*buffer_size_az+1:v*buffer_size_az;
  grid_array_az(buffer,3) = repmat(v,length(buffer),1);
end

if (rem_buffer_az ~= 0)
  buffer = v*buffer_size_az+1:Ng_az;
  grid_array_az(buffer,3) = repmat(v+1,length(buffer),1);
  Nbuffers_az = Nbuffers_az + 1; % add remaining buffer
end

grid_array_r = repmat(NaN,Ng_r,3);
grid_array_r(:,1) = [1 offset_r+Npixels_g+1:Npixels_g:offset_r+(Ng_r-
1)*Npixels_g+1]';
grid_array_r(:,2) = [offset_r+Npixels_g:Npixels_g:offset_r+ ...
                      (Ng_r-1)*Npixels_g Npixels]';

for v = 1:Nbuffers_r
  buffer = (v-1)*buffer_size_r+1:v*buffer_size_r;
  grid_array_r(buffer,3) = repmat(v,length(buffer),1);
end

if (rem_buffer_r ~= 0)
  buffer = v*buffer_size_r+1:Ng_r;
  grid_array_r(buffer,3) = repmat(v+1,length(buffer),1);
  Nbuffers_r = Nbuffers_r + 1; % add remaining buffer
end

% output to screen
```

```
fprintf(1,'\n');
fprintf(1,'Data devided in %3.0f buffers of %3.0f grid cells in azimuth
direction\n', ...
        Nbuffers_az,buffer_size_az);
if (rem_buffer_az ~= 0)
  fprintf(1,'plus a remaining buffer of %3.0f grid cells\n', ...
          rem_buffer_az);
end
fprintf(1,'and in %3.0f buffers of %3.0f grid cells in range direction\n', ...
        Nbuffers_r,buffer_size_r);
if (rem_buffer_az ~= 0)
  fprintf(1,'plus a remaining buffer of %3.0f grid cells\n', ...
          rem_buffer_r);
end
fprintf(1,'\n');


%determine number of calibration points per grid cell
Ncal_grid = max(1,round(Ncal/(Ng_az*Ng_r)));


% --------------------------------------------------------------------
% Main loop per buffer
% --------------------------------------------------------------------

grid_count = 0;
cal_amp = repmat(NaN,Ng_az*Ng_r*Ncal_grid,Nslc_selected);
cal_mean_amp = repmat(NaN,Ng_az*Ng_r*Ncal_grid,1);
cal_std_amp = repmat(NaN,Ng_az*Ng_r*Ncal_grid,1);

for v = 1:Nbuffers_az


  % ------------------------------------------------------------------
  % Read data
  % ------------------------------------------------------------------

  index_az = find(grid_array_az(:,3)==v);
  Ngrid_az = length(index_az);
  begin_buffer_az = grid_array_az(index_az(1),1);
  end_buffer_az = grid_array_az(index_az(end),2);
  buffer_size_az = end_buffer_az-begin_buffer_az+1;

  for w = 1:Nbuffers_r

    fprintf(1,'Processing buffer %3.0f in azimuth and buffer %3.0f in
range\n',v,w)

    index_r = find(grid_array_r(:,3)==w);
    Ngrid_r = length(index_r);
    begin_buffer_r = grid_array_r(index_r(1),1);
    end_buffer_r = grid_array_r(index_r(end),2);
    buffer_size_r = end_buffer_r-begin_buffer_r+1;

    slc_array = repmat(NaN,[buffer_size_az buffer_size_r Nslc]);

    for z = 1:Nifgs
      Nlines_file = crop_in(z,2)-crop_in(z,1)+1;
      loffset = crop(1)-crop_in(z,1);
      poffset = crop(3)-crop_in(z,3);
      slc_array(:,:,z) =
freadbk(char(filenames_slc(z)),Nlines_file,'cpxfloat32',...
                          begin_buffer_az+loffset,end_buffer_az+loffset,...
                          begin_buffer_r+poffset,end_buffer_r+poffset);
```

```
    end

    if Nslc == Nifgs+1
      % read last slc
      Nlines_file = crop_in(Nslc,2)-crop_in(Nslc,1)+1;
      loffset = crop(1)-crop_in(Nslc,1);
      poffset = crop(3)-crop_in(Nslc,3);

      fileinfo = dir(char(filenames_slc(Nslc)));
      Nel = (crop_in(Nslc,2)-crop_in(Nslc,1)+1)*(crop_in(Nslc,4)-
crop_in(Nslc,3)+1);
      if fileinfo.bytes==Nel*2*4
        master_format = 'cpxfloat32';
      elseif fileinfo.bytes==Nel*2*2;
        master_format = 'cpxint16';
      else
        error('Something is wrong with the file size of the master SLC.');
      end

      slc_array(:,:,Nslc) =
freadbk(char(filenames_slc(Nslc)),Nlines_file,master_format,...
                              begin_buffer_az+loffset,end_buffer_az+loffset,...
                              begin_buffer_r+poffset,end_buffer_r+poffset);
    end

    if ischar(filename_water_mask)

      Nel_master = (crop_in(Nslc,2)-crop_in(Nslc,1)+1)*(crop_in(Nslc,4)-
crop_in(Nslc,3)+1);
      Nel_mrm = (crop(2)-crop(1)+1)*(crop(4)-crop(3)+1);

      fileinfo = dir(filename_water_mask);

      if fileinfo.bytes==Nel_master
        Nlines_file = crop_in(Nslc,2)-crop_in(Nslc,1)+1;
        loffset = crop(1)-crop_in(Nslc,1);
        poffset = crop(3)-crop_in(Nslc,3);
      elseif fileinfo.bytes==Nel_mrm
        Nlines_file = crop(2)-crop(1)+1;
        loffset = 0;
        poffset = 0;
      end

      water_mask = freadbk(filename_water_mask,Nlines_file,'uint8',...
                           begin_buffer_az+loffset,end_buffer_az+loffset,...
                           begin_buffer_r+poffset,end_buffer_r+poffset);

      land_index = find(water_mask==0);
      if length(land_index)>100
        land_flag = 1; % skip selection if only water in buffer
      else
        land_flag = 0;
      end
    else
      land_flag = 1;
    end


    % ---------------------------------------------------------------------
    % Amplitude calibration
    % ---------------------------------------------------------------------

    amp_array = abs(slc_array);
    mean_amp = nanmean(amp_array(:,:,slc_selection),3);
```

```
    if land_flag

      if ischar(filename_water_mask)
        [water_az,water_r] = find(water_mask);
        for k = 1:length(water_az)
          amp_array(water_az(k),water_r(k),:) = NaN; %added to mask water bodies
        end
        mean_amp = nanmean(amp_array(:,:,slc_selection),3);
      end

      std_amp = nanstd(amp_array(:,:,slc_selection),0,3);
      % the 0-flag causes std to normalize by (N-1)
      amp_disp = std_amp./mean_amp; % amplitude dispersion

      for g = 1:Ngrid_az
        for h = 1:Ngrid_r
          grid_count = grid_count+1;

          begin_grid_az = grid_array_az(index_az(g),1)-begin_buffer_az+1;
          end_grid_az = grid_array_az(index_az(g),2)-begin_buffer_az+1;
          begin_grid_r = grid_array_r(index_r(h),1)-begin_buffer_r+1;
          end_grid_r = grid_array_r(index_r(h),2)-begin_buffer_r+1;

          amp_disp_grid =
amp_disp(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r);
          mean_amp_grid =
mean_amp(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r);
          std_amp_grid =
std_amp(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r);

        [sorttemp,sortind]=sort(amp_disp_grid(:));
        sortind = sortind(1:Ncal_grid);

        for k = 1:Nslc_selected
            amp_temp =
amp_array(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r,slc_selection(k));
            cal_amp((grid_count-1)*Ncal_grid+1:grid_count*Ncal_grid,k) =
amp_temp(sortind)';
          end
          cal_mean_amp((grid_count-1)*Ncal_grid+1:grid_count*Ncal_grid) =
mean_amp_grid(sortind);
          cal_std_amp((grid_count-1)*Ncal_grid+1:grid_count*Ncal_grid) =
std_amp_grid(sortind);

        end
      end

    end

  end
end

cal_index = find(~isnan(cal_amp(:,1)));
cal_amp = cal_amp(cal_index,:);
cal_mean_amp = cal_mean_amp(cal_index);
cal_std_amp = cal_std_amp(cal_index);

calfactors = empcal_nonlin_Q(cal_amp,cal_std_amp.^2,cal_mean_amp);
```